INSTITUTO TÉCNICO RICALDONE · ESPECIALIDAD DE DESARROLLO DE SOFTWARE

Repaso para la Defensa Externa de PetCore

Sistema de gestión veterinaria: sistema web (HTML, CSS, Bootstrap, JavaScript sin frameworks), API (Java, Spring Boot, JPA/Hibernate), base de datos Oracle Cloud y app móvil Android/iOS con Capacitor.

Frontend → Vercel (petcore-seven.vercel.app)

API / Backend → Heroku (petcore-8afada45fabc.herokuapp.com)

Base de datos → Oracle Cloud, gestionada con SQL Developer

App móvil → Capacitor (Android / iOS)

Contenido

1. Rúbrica oficial de evaluación externa
2. Verificación técnica de último momento
3. Arquitectura del Backend (API Java / Spring Boot)
4. Arquitectura del Frontend y la App Móvil
5. Base de datos (Oracle Cloud)
6. Roles, autenticación, CRUD y alcance del Propietario en móvil
7. Conceptos clave explicados (23 temas)
8. Banco de preguntas por dificultad (42 preguntas)
9. Logística del equipo (6 integrantes, 4 días, 2 por turno)
10. Guion para el Jurado Externo
11. Guion para el Público

SECCIÓN 1

Rúbrica oficial de evaluación externa

Estos son los 10 criterios reales que evalúa el jurado. Úsalos como checklist de preparación del equipo.

- [ ] **1. Documentación del sistema.** Manual de usuario con imágenes que muestre cómo se realizan los procesos + documentación técnica para poner en marcha el software + diagramas que expliquen el funcionamiento.
- [ ] **2. Diseño de las interfaces web.** Todas las interfaces ajustan su disposición según el tamaño de pantalla; fácil interacción, comprensibles, con diseño estético y profesional.
- [ ] **3. Diseño de las interfaces móviles.** Mismo criterio que el anterior, aplicado a la app móvil.
- [ ] **4. Funcionalidad del sistema web.** Todos los mantenimientos (CRUD) funcionan correctamente y están validados en el Frontend y en el Backend.
- [ ] **5. Funcionalidad de la aplicación móvil.** Igual que el criterio 4, pero en la app móvil.
- [ ] **6. Validaciones e integridad de los datos.** Se valida que los datos correspondan al tipo de información solicitado (campos numéricos, fechas, textos, imágenes) tanto en el sistema web como en la app móvil, verificado por la API.
- [ ] **7. Autenticación y roles de usuario.** El sistema tiene inicio de sesión y demuestra claramente el manejo de roles a nivel de Frontend y Backend, y las funciones que cada uno puede realizar.
- [ ] **8. Protección de endpoints y vistas.** El diagrama de casos de uso evidencia la implementación de autenticación/autorización; demostración desde el frontend de endpoints públicos y protegidos; se bloquea el botón "atrás" del navegador tras cerrar sesión.
- [ ] **9. Despliegue del Backend, sistema web y móvil.** La API está desplegada y configurada para recibir peticiones de orígenes definidos (CORS); el sistema web es accesible por una dirección pública; la app móvil está instalada en un dispositivo físico Android.
- [ ] **10. Dominio.** El equipo responde con seguridad, demuestra dominio de la temática, usa lenguaje técnico y descripciones acertadas del trabajo realizado.

> Orientación oficial del jurado: Frontend en HTML/CSS/Bootstrap/JavaScript sin frameworks, desplegado en Vercel; Backend en Java + Spring Boot + JPA/Hibernate, desplegado en Heroku; base de datos en Oracle Cloud gestionada con SQL Developer.

SECCIÓN 2 · CRITERIO 9

Verificación técnica de último momento

No basta con que el sistema haya funcionado la semana pasada — Vercel y Heroku son servicios externos que pueden caerse, expirar un dominio gratuito, o simplemente tener un "cold start" largo el día de la defensa. Verifíquenlo **en vivo, justo antes** de presentar, en cualquiera de los 8 turnos.

- [ ] **Vercel activo (dominio 1).** Abrir `petcore-seven.vercel.app` y confirmar que carga el login, no un error 404/expirado.
- [ ] **Vercel activo (dominio 2).** Lo mismo con `petcore-oerh.vercel.app` si tu equipo usa ambos dominios.
- [ ] **API de Heroku despierta.** Cargar el login unos minutos antes para evitar el "cold start" largo justo delante del jurado.
- [ ] **Login de cada rol funciona en producción.** Probar Administrador, Veterinario, Recepcionista y Farmacéutico contra la URL pública (no localhost).
- [ ] **App móvil instalada y probada.** Confirmar que está instalada en el dispositivo Android físico que se usará, con sesión y datos de ejemplo listos.
- [ ] **Batería e internet del dispositivo móvil.** Cargado, con datos móviles o WiFi del lugar confirmado — no asuman que el WiFi de la expo va a funcionar igual que en casa.

SECCIÓN 3 · BACKEND · JAVA + SPRING BOOT

Arquitectura de la API de PetCore

Monolito modular en Java 17 con Spring Boot 4.1, JPA/Hibernate sobre Oracle, desplegado en Heroku. Paquete raíz `PTC_PETCORE.API`.

Origen del proyecto (Spring Initializr)

El `pom.xml` hereda de `spring-boot-starter-parent` (v4.1.0), lo que confirma que el proyecto nació desde **start.spring.io** (Spring Initializr), con Maven como gestor de dependencias y Java 17.

- **Group:** PTC_PETCORE · **Artifact:** API · **Packaging:** Jar
- **Java:** 17 (fijado también en `system.properties` para Heroku)
- **Build tool:** Maven (`pom.xml`, wrapper `mvnw` / `mvnw.cmd`)
- Incluye `ServletInitializer.java`: permite empacar también como WAR, aunque se despliega como Jar ejecutable en Heroku.

Dependencias de la API y para qué sirve cada una

DEPENDENCIA	PARA QUÉ SE USA EN PETCORE
`spring-boot-starter-data-jpa`	JPA/Hibernate: mapea las 50 tablas Oracle a entidades Java y genera los repositorios.
`spring-boot-starter-web/-webmvc`	Spring MVC: controllers REST, serialización JSON automática.
`spring-boot-starter-security`	Autenticación/autorización: filtra cada request, aplica roles.
`spring-security-crypto`	Utilidades criptográficas usadas junto al hashing Argon2 de contraseñas.
`jjwt-api / impl / jackson`	Firma y valida los JWT guardados en la cookie `authToken`.
`spring-boot-starter-validation`	Bean Validation (`@NotBlank`, `@Email`, `@Pattern`...) sobre los DTO de entrada.
`spring-boot-starter-websocket`	Soporte WebSocket de bajo nivel para el endpoint `/ws`.
`spring-boot-starter-mail`	Envío de correos: código de recuperación de contraseña.
`google-api-client`	Verifica el ID Token de "Iniciar sesión con Google" del lado del servidor.
`cloudinary-http44`	Sube y gestiona imágenes (fotos de perfil, mascotas).
`bcprov-jdk18on` (BouncyCastle)	Cifrado ECDH/VAPID de Web Push.
`web-push`	Notificaciones push al navegador (Web Push API + VAPID).
`firebase-admin`	Push nativo a la app móvil vía Firebase Cloud Messaging.
`ojdbc11`	Driver JDBC oficial de Oracle.
`lombok`	Genera getters/setters/toString por anotación.
`dotenv-java`	Carga variables de entorno desde `.env` en desarrollo local.

DEPENDENCIA	PARA QUÉ SE USA EN PETCORE
`spring-boot-starter-tomcat`	Servidor embebido que ejecuta la API.

Arquitectura modular por feature (no por capa)

PetCore usa **51 módulos verticales** (uno por entidad/tabla, más módulos transversales como `auth` y `push`) bajo `modules/`. Cada módulo repite siempre 6 carpetas:

- **controller/** — clases `@RestController`, reciben el HTTP request y devuelven `ResponseEntity<ApiResponse<T>>`.
- **service/** — lógica de negocio; nunca usa try/catch, lanza excepciones propias del módulo.
- **repository/** — interfaces `JpaRepository<Entidad, Long>`.
- **entity/** — clases `@Entity` mapeadas a las tablas `TBL_*`, con Lombok y relaciones JPA.
- **dto/** — objetos de transferencia con Bean Validation.
- **exceptions/** — 5 excepciones propias por módulo heredando de clases globales.

Paquetes transversales

PAQUETE	RESPONSABILIDAD
`security/config`	`SecurityConfig.java` (1219 líneas) — toda la matriz de autorización por rol y endpoint.
`security/cors`	`CorsConfig.java` — orígenes, métodos y headers permitidos.
`security/jwt`	`JWTUtils`, `JwtCookieAuthFilter`, `AuthenticatedPrincipal`, `RestAuthHandlers`.
`exceptions/global`	`GlobalExceptionHandler`: traduce cualquier excepción a una respuesta JSON consistente.
`config/websocket`	`WebSocketConfig` + `EventoWebSocketHandler`: endpoint `/ws`.
`config/cloudinary`	Credenciales y cliente de subida de imágenes.
`config/crypto`	`Argon2PasswordConfig`: algoritmo de hashing de contraseñas.
`config/paginacion`	Helper para las búsquedas paginadas.
`config/async`	Ejecuta tareas (ej. notificaciones) sin bloquear el hilo HTTP.
`response`	`ApiResponse<T>`: envoltorio estándar `{success, message, data}` de todas las respuestas.

> El módulo `auth/` es especial, no sigue el patrón CRUD: contiene `AuthController`, `ContraseniaController`, `RecuperacionController`, el verificador de Google y la recuperación de contraseña por correo.

CORS: ¿quién puede llamar a la API?

Definido en `security/cors/CorsConfig.java`, un único bean `CorsConfigurationSource` registrado para `/**`.

- **Orígenes permitidos:** localhost/127.0.0.1 (desarrollo), `*.herokuapp.com`, `petcore-seven.vercel.app` y `petcore-oerh.vercel.app`, `localhost:5173` (Vite), `https://localhost` y `capacitor://localhost` (apps nativas), más orígenes extra vía variable de entorno `security.cors.extra-origins`.
- **allowCredentials = true** — obligatorio porque la sesión viaja en cookie, no en header.

- **Métodos:** GET, POST, PUT, PATCH, DELETE, OPTIONS.
- **Headers permitidos:** Origin, Content-Type, Accept, Authorization, X-Requested-With, Cookie...
- **Headers expuestos:** Set-Cookie, Cookie, Authorization, Content-Disposition.
- **maxAge:** 3600s.

> **Detalle fino:** `WebSocketConfig.java` repite manualmente la misma lista de orígenes para el handshake de `/ws`, porque Spring Security no comparte automáticamente la configuración de CORS HTTP con WebSocket.

Spring Security: autenticación, roles y "deny by default"

`security/config/SecurityConfig.java` (1219 líneas) — el archivo más importante de la API.

- CSRF deshabilitado (API stateless con JWT, sin formularios HTML tradicionales).
- `sessionCreationPolicy(STATELESS)` — no hay sesión guardada en servidor.
- `RestAuthHandlers` como `authenticationEntryPoint` y `accessDeniedHandler` → respuestas 401/403 en JSON consistente.
- `JwtCookieAuthFilter` insertado antes de `UsernamePasswordAuthenticationFilter`.

Roles del sistema: `ROLE_Administrador` `ROLE_Veterinario` `ROLE_Recepcionista` `ROLE_Farmaceutico` y `ROLE_Propietario` (dueño de mascotas, app móvil). Constante `STAFF = {ADMIN, VET, FARM, RECEP}` y helper `staffMas(extra...)`.

El patrón clave: "deny by default"

Cada tabla tiene su método `reglasX(auth)` con reglas explícitas por endpoint. Al final de toda la cadena:

```
auth.anyRequest().denyAll();
```

Cualquier ruta nueva sin regla explícita queda bloqueada automáticamente — es el punto de seguridad más fuerte para mencionar en la defensa.

Ejemplos reales de reglas por endpoint

MÉTODO	RUTA	QUIÉN ACCEDE
GET	`/api/cfgs_sistema`	Público (pinta el login antes de autenticar)
POST	`/api/empleados/primer-administrador`	Público — solo mientras no hay ningún empleado (bootstrap)
GET	`/api/push/vapid-public-key`	Público — clave VAPID necesaria antes del login
POST	`/api/auth/login/empleado`	Público (es el propio login)
GET	`/api/alergia_mascota/por_mascota/{id}`	Veterinario y Propietario
POST	`/api/citas`	Recepcionista y Propietario
PATCH	`/api/citas/{id}/estado`	Veterinario y Recepcionista
GET	`/api/empleados/**`	Solo Administrador
—	`/ws/**`	Público (handshake abierto, mejora pendiente)

> Debilidad honesta a mencionar si preguntan: el handshake de `/ws/**` está en `permitAll()` a propósito — el canal WebSocket aún no valida el JWT en la conexión inicial.

JWT firmado + cookie httpOnly

`security/jwt/JWTUtils.java` : firma con **HMAC-SHA256**, secreto en variable de entorno `SECRET_KEY_JWT`. Claims: `id` , `rol` , `tipo` ("EMPLEADO"/"PROPIETARIO"), `sub` =correo, `iss` , `exp` .

`security/jwt/JwtCookieAuthFilter.java` : filtro `OncePerRequestFilter` , lee la cookie `authToken` (empleados) o `authTokenPropietario` (dueños). **Sesión deslizante:** cada request válido renueva la cookie (TTL 900s = 15 min).

Dos audiencias, dos cookies: **empleados** (portal web) y **propietarios** (app móvil), cada uno con su propio login y endpoint `/api/auth/me/...` , firmados con la misma utilidad pero mantenidos separados.

Entidades JPA y relaciones principales

- **EmpleadosEntity** (`TBL_EMPLEADOS`): "hub" del sistema. `@ManyToOne` a Roles (obligatorio) y Turnos (opcional). ~17 relaciones `@OneToMany` inversas (Expedientes, Citas, Ventas, Hospitalización, etc.).
- **RolesEntity** (`TBL_ROLES`): catálogo simple de los 5 roles.
- **PropietariosEntity** (`TBL_PROPIETARIOS`): usuarios del portal móvil, incluye `pro_google_id` .
- **MascotasEntity** (`TBL_MASCOTAS`): `@ManyToOne` a Raza, Pelaje y Propietario; `@OneToMany` a Citas, Cartillas, Expediente, Hospitalización, etc.

> Convención técnica: relaciones directas con `fetch=LAZY` , PKs con `GenerationType.IDENTITY` (compatible con Oracle), y `hibernate.default_batch_fetch_size=50` para evitar el problema N+1.

Validaciones e integridad de datos

Bean Validation en los DTO, ejemplo real de `EmpleadosDTO` :

```
@NotBlank
@Email
private String empCorreo;

@Pattern(regexp = "^[0-9]{8}-[0-9]$")
private String empDui;

@Past
private LocalDate empFechaNac;

@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
private String empContrasenia;
```

`exceptions/global/GlobalExceptionHandler.java` centraliza el manejo de errores: `MethodArgumentNotValidException` → 400, `NotFoundException` → 404, `DuplicateException` / `DataIntegrityViolationException` → 409, `BadCredentialException` → 401, `CuentaInactivaException` → 403, `Exception` genérico → 500.

WebSockets en el Backend

WebSocket **puro/crudo** (sin STOMP, sin broker de mensajes). `config/websocket/WebSocketConfig.java` registra el endpoint `/ws`; `EventoWebSocketHandler.java` mantiene todas las sesiones conectadas y expone `difundir(evento, datos)` que hace broadcast total a todas las sesiones.

- Ping cada 25s (evita que Heroku corte conexiones inactivas a los 55s).
- Pool de hilos dedicado para que un broadcast lento no bloquee el hilo HTTP.
- Uso: campanita de notificaciones, indicador de conectado/desconectado, actualización en caliente de citas/asistencias/configuración.

Configuración y despliegue en Heroku

Todo externalizado a variables de entorno: BD, JWT, cookies, Cloudinary, VAPID, Firebase, SMTP, Google OAuth. Pool HikariCP máx. 8 conexiones, Tomcat máx. 30 hilos (ajustado al plan gratuito).

```
web: java -Dserver.port=$PORT $JAVA_OPTS -jar target/API-0.0.1-SNAPSHOT.jar
```

Dominio real: `petcore-8afada45fabc.herokuapp.com`. Un workflow de GitHub Actions (`keep-alive.yml`) hace ping periódico para reducir el "cold start" del dyno gratuito.

SECCIÓN 4 · FRONTEND · HTML/CSS/JS PURO + APP CAPACITOR

Arquitectura del Frontend y la App Móvil

HTML, CSS, Bootstrap 5.3.8 y JavaScript sin frameworks, desplegado en Vercel. La app móvil usa Capacitor para empaquetar ese mismo frontend dentro de un WebView nativo Android/iOS.

Patrón Controller / Service, páginas por rol

- **js/controllers/** (~69 archivos) — un controller por página, maneja eventos y pinta datos en el DOM.
- **js/services/** (~136 archivos) — un service por entidad; son los únicos que hacen `fetch` a la API.
- **js/utils/** — helpers compartidos: alertas (SweetAlert2), fechas, selects/datepickers, lector de código de barras, generación de PDF.

pages/ organizado por rol

- **Administrador** 20 páginas: personal, inventario, compras, citas, servicios, turnos, reportes, configuración...
- **Veterinario** 16 páginas: expedientes clínicos, citas, cuidados, alergias, hospitalización, posología, reportes...
- **Recepcionista** 9 páginas: citas, sala de espera, registro de mascotas/propietarios, catálogos...
- **Farmacéutico** 7 páginas: punto de venta, stock, posología, registro de ventas...

Autenticación: sesión por cookie, no por localStorage

Dato clave: el proyecto migró de JWT-en-localStorage a cookie httpOnly. El comentario en `login_controller.js` dice textualmente: *"ya no se guarda nada en localStorage/sessionStorage aquí"*.

1. **Login:** `login_controller.js` → `AuthModule.iniciarSesion()` llama a `login_service.js`, que hace `POST /api/auth/login/empleado`.
2. **Cookie httpOnly:** el backend pone la cookie `authToken`; JavaScript nunca puede leerla ni manipularla.
3. **credentials: "include":** `fetch_con_credenciales.js` parchea `window.fetch` globalmente para mandar siempre las cookies.
4. **Verificación de sesión:** `sesion_service.js` → `verificarSesionSync()` hace un XHR síncrono contra `GET /api/auth/me/empleado`.
5. **Logout:** `POST /api/auth/logout` (el servidor borra la cookie); el frontend también limpia `sessionStorage` y el Cache Storage.

> ¿Qué SÍ hay en localStorage? Solo preferencias de interfaz sin información sensible: tema claro/oscuro, animaciones, opacidad de fondo, formato de hora. Cero tokens, cero contraseñas.

Protección de rutas, roles y el botón "atrás"

```
const sesion = SesionService.verificarSesionSync();
if (!sesion) location.replace('../Cuentas/login.html');
if (!config.rolesPermitidos.includes(sesion.rol)) {
  location.replace('../acceso_denegado.html');
}
```

```
window.addEventListener("pageshow", (e) => {
  if (e.persisted) location.reload();
});
```

Cuando el navegador restaura una página desde el bfcache tras pulsar "Atrás", esto fuerza una recarga real que vuelve a ejecutar la verificación de sesión.

Interceptor global de fetch

`js/controllers/fetch_con_credenciales.js` agrega: `credentials: "include"` siempre; reintentos con backoff ante 502/503/504; desenvoltura automática del `ApiResponse`; logout automático si una llamada responde 401/403 fuera de `/api/auth/`.

WebSocket en el Frontend

`js/services/websocket_service.js` : conexión nativa hacia `wss://petcore-8afada45fabc.herokuapp.com/ws`. Heartbeat manual, reconexión con backoff exponencial + jitter. Se usa para la campanita de notificaciones, el indicador de conexión y actualizaciones en vivo.

La app móvil: Capacitor

Carpeta `PetCore-APP-Oficial/`. Capacitor toma el mismo frontend web y lo empaqueta dentro de un WebView nativo.

- `appId`: `com.petcore.app.oficial`, `webDir`: `dist` (carpeta que genera Vite).
- Plugin `CapacitorCookies` habilitado — imprescindible para que la cookie httpOnly funcione en el WebView nativo.
- Plugin `CapacitorHttp` habilitado — las peticiones pasan por el puente nativo.
- `SocialLogin` (Google habilitado) para login con Google desde la app.

Compilación para Android

PASO	COMANDO	QUÉ HACE
1	`npm run build`	Vite compila el frontend a `dist/`
2	`npx cap sync android`	Copia `dist/` al proyecto Android y sincroniza plugins
3	Android Studio / `gradlew`	Compila el APK/AAB (`minSdkVersion 24` , `targetSdk 36`)
4	Instalación física	App instalada en un dispositivo Android físico

SECCIÓN 5 · BASE DE DATOS · ORACLE CLOUD

Modelo de datos de PetCore

50 tablas en Oracle gestionadas con SQL Developer, estándar `SCREAMING_SNAKE_CASE`: `TBL_<ENTIDAD>`, constraints nombradas `PK_/FK_/UQ_/CK_/IX_`.

Estándar de nombres

ELEMENTO	CONVENCIÓN	EJEMPLO
Tabla	`TBL_<ENTIDAD>`	`TBL_EMPLEADOS`
Columna	`<PREFIJO>_<CAMPO>`	`EMP_CORREO`
Llave primaria	`PK_<CAMPO>`	`PK_EMP_ID`
Llave foránea	`FK_<ORIGEN>_<DESTINO>`	`FK_EMP_ROL`
Único	`UQ_<CAMPO>`	`UQ_EMP_CORREO`
Check	`CK_<CAMPO>`	`CK_EMP_DUI` (regex `00000000-0`)

TBL_EMPLEADOS: la tabla de usuarios del sistema

COLUMNA	TIPO / REGLA	PARA QUÉ
`EMP_CORREO`	UNIQUE + CHECK regex email	Usuario de login y avisos
`EMP_CONTRASENIA`	VARCHAR2(300)	Hash Argon2 — nunca texto plano
`EMP_DUI`	CHAR(10) + CHECK regex	DUI de El Salvador, formato `00000000-0`
`EMP_ROL_ID`	FK → TBL_ROLES	Rol del empleado
`EMP_TUR_ID`	FK → TBL_TURNOS (nullable)	Turno asignado
`EMP_ESTADO`	CHECK IN ('Activo','Inactivo','Suspendido')	Baja lógica

Las 50 tablas, por dominio funcional

Catálogos y personal

`TBL_ROLES` `TBL_TURNOS` `TBL_EMPLEADOS` `TBL_CFG_SISTEMA`

Catálogos clínicos y mascotas

`TBL_ESPECIE` `TBL_RAZA` `TBL_PELAJE` `TBL_PROPIETARIOS` `TBL_MASCOTAS` `TBL_ALERGIAS` `TBL_ALERGIA_MASCOTA` `TBL_DIAGNOSTICOS` `TBL_DIAGNOSTICO_RAZA` `TBL_MASCOTA_ENFERMEDAD`

Citas y atención clínica (el núcleo del sistema)

`TBL_SERVICIOS` `TBL_CITAS` `TBL_CITA_SERVICIO` `TBL_EXPEDIENTE` `TBL_DIAGNOSTICO_EXPEDIENTE` `TBL_EXPEDIENTE_SERVICIO` `TBL_PRESCRIPCION` `TBL_DETALLE_PRESCRIPCION` `TBL_HOSPITALIZACION`

`TBL_EVOLUCION_CLINICA` `TBL_TAREAS_MEDICAS` `TBL_CARTILLAS` `TBL_ANTIPARASITARIO` `TBL_HISTORIAL_TRATAMIENTO` `TBL_CUIDADOS` `TBL_POSOLOGIA_FARMACO`

Inventario, compras y ventas (farmacia)

`TBL_PROVEEDOR` `TBL_TIPO_PRODUCTO` `TBL_MARCA` `TBL_PRODUCTO` `TBL_COMPRAS` `TBL_DETALLE_COMPRA` `TBL_SOLICITUDES_BODEGA` `TBL_DETALLE_SOLICITUD` `TBL_VENTAS` `TBL_DETALLE_VENTA` `TBL_MOVIMIENTO_INVENTARIO`

Comunicación y notificaciones

`TBL_ANUNCIOS` `TBL_ANUNCIO_REACCIONES` `TBL_ANUNCIO_COMENTARIOS` `TBL_NOTIFICACIONES` `TBL_NOTIFICACION_DESTINATARIO` `TBL_SUSCRIPCIONES_PUSH` `TBL_PUSH_TOKEN_FCM`

Recursos humanos

`TBL_ASISTENCIA_EMPLEADOS` `TBL_DIAS_FERIADOS`

Relaciones clave: el flujo clínico completo

```
PROPIETARIO → MASCOTA → CITA → CITA_SERVICIO → EXPEDIENTE → DIAGNOSTICO_EXPEDIENTE → PRESCRIPCION
→ DETALLE_PRESCRIPCION → MOVIMIENTO_INVENTARIO
```

SECCIÓN 6 · ROLES, AUTENTICACIÓN Y CRUD

Roles, autenticación y funcionalidad CRUD

ROL	ENFOQUE	MÓDULOS PRINCIPALES
Administrador	Control total	Personal, inventario, compras, servicios, turnos, días feriados, asistencias, reportes, configuración
Veterinario	Atención clínica	Expedientes, citas, diagnósticos, prescripciones, hospitalización, cuidados, alergias, catálogos clínicos
Recepcionista	Front desk	Registro de propietarios/mascotas, agenda de citas, sala de espera
Farmacéutico	Farmacia / POS	Punto de venta, stock, posología, registro de ventas
Propietario	App móvil	Ver sus mascotas, agendar citas, ver historial clínico (solo lectura)

Matriz de funcionalidad CRUD por rol (personal)

MÓDULO	ADMIN	VETERINARIO	RECEPCIONISTA	FARMACÉUTICO
Personal (empleados)	CRUD	—	—	—
Inventario / productos	CRUD	Leer	—	Leer / Stock
Compras a proveedor	CRUD	—	—	—
Servicios y turnos	CRUD	Leer	Leer	—
Propietarios y mascotas	Leer	Leer	CRUD	—
Citas	Gestionar	Atender / estado	Crear / agendar	—
Expedientes clínicos	—	CRUD	—	—
Diagnósticos y prescripciones	—	CRUD	—	Leer (para despachar)
Hospitalización	—	CRUD	—	—
Punto de venta (POS)	—	—	—	CRUD
Reportes	Generales	Clínicos	—	—
Configuración del sistema	Editar	Leer	Leer	Leer

Alcance CRUD real del rol Propietario en la app móvil

El Propietario es intencionalmente limitado: consume información clínica, no la genera ni la edita. Verificado contra `SecurityConfig.java`:

MÓDULO	LO QUE SÍ PUEDE HACER	LO QUE NO PUEDE HACER (Y POR QUÉ)
Mascotas	Ver sus propias mascotas; actualizar solo la **foto**	Registrar una mascota nueva ni editar sus datos clínicos — requiere verificación presencial del personal.

MÓDULO	LO QUE SÍ PUEDE HACER	LO QUE NO PUEDE HACER (Y POR QUÉ)
Citas	Ver sus citas; **crear** una cita nueva	Editar o cancelar una cita ya creada, ni cambiar su estado — lo gestiona Recepcionista/Veterinario.
Expediente clínico	Ver su historial médico (solo lectura)	Editar diagnósticos, notas clínicas o tratamientos.
Cartillas / Antiparasitarios	Ver (solo lectura)	Editar o eliminar un registro.
Cuidados (contenido educativo)	Ver recomendaciones	Editar el catálogo — lo mantiene el personal clínico.
Ventas propias	Ver su historial de compras en farmacia	Editar o eliminar una venta.
Notificaciones	Ver las propias; marcarlas como leídas	Ver o modificar notificaciones de otro usuario.
Perfil propio	Ver y autoeditar sus datos, cambiar contraseña, desvincular Google	Editar el perfil de otro propietario.

> Frase lista para la defensa: "*Esa acción está reservada al personal de la clínica por diseño, no es una omisión — el propietario es quien consume la información clínica y agenda citas, pero quien la gestiona y modifica siempre es el personal autorizado. Está así desde la configuración de seguridad del backend.*"

Cómo demostrar "endpoints públicos vs. protegidos"

1. **Endpoint público:** `GET /api/cfgs_sistema` responde 200 sin sesión.
2. **Endpoint protegido sin sesión:** intentar `GET /api/empleados` sin cookie válida responde **401**.
3. **Rol equivocado:** navegar a una URL de otro rol redirige a `acceso_denegado.html`; por API responde **403**.
4. **Botón "Atrás":** la página se recarga y vuelve a pedir login gracias al listener de `pageshow`.

SECCIÓN 7 · LO QUE MÁS PREGUNTAN EN LA DEFENSA INTERNA

Conceptos clave, explicados con el código real

¿Qué es CORS y cómo está configurado en tu API?

Mecanismo del navegador que decide si un sitio puede hacer peticiones a una API en otro dominio. PetCore lo configura en `CorsConfig.java` con lista blanca de orígenes y `allowCredentials=true` (obligatorio por la cookie de sesión). "CORS no es seguridad contra atacantes — la seguridad real la decide Spring Security en el backend."

¿Qué son los WebSockets y para qué los usas?

Canal bidireccional persistente entre navegador y servidor. PetCore implementa WebSocket puro (sin STOMP) con broadcast total a todas las sesiones. Uso: notificaciones en tiempo real, indicador de conexión, actualización en vivo.

¿Qué guardas en localStorage? ¿Por qué no el token?

localStorage es accesible desde cualquier JavaScript (incluido XSS). El token vive en una cookie httpOnly inaccesible a JS. localStorage solo guarda preferencias de interfaz sin información sensible.

¿Cómo funciona tu JWT? ¿Por qué en cookie y no en header?

JWT firmado entregado dentro de una cookie httpOnly, combinando stateless con inaccesibilidad a JavaScript. Sesión deslizante: cada request renueva la cookie (15 min).

Si usan cookies, ¿cómo evitan CSRF?

Cookie con `SameSite` + `Secure`, CORS limitando orígenes, y API JSON pura reduce la superficie de ataque clásica.

¿Cómo bloquean el botón "Atrás" después de cerrar sesión?

Evento `pageshow`: si `event.persisted` es true, fuerza un `reload()` que vuelve a ejecutar el guard de sesión.

¿Cómo protegen tantos endpoints sin equivocarse?

Patrón "deny by default": `auth.anyRequest().denyAll()` al final — un endpoint nuevo sin regla queda bloqueado automáticamente.

¿Cómo funciona el login con Google?

`GoogleTokenVerifier.java` verifica el ID Token directamente con los servidores de Google antes de crear/vincular el `PropietariosEntity`.

¿Es PetCore una PWA? ¿Qué hace el Service Worker?

Parcialmente. `sw.js` solo maneja Web Push (no cachea assets estáticos). El push nativo de la app usa Firebase Cloud Messaging por separado.

¿Dónde se valida realmente un dato: frontend o backend?

En ambos, pero la garantía real está en el backend (Bean Validation + GlobalExceptionHandler) — nunca confiar solo en la validación del frontend.

¿Por qué no hicieron la app móvil "nativa" desde cero?

Capacitor empaqueta el frontend web en un WebView nativo con plugins — un solo código fuente para web y móvil.

¿Qué significa que la API sea "RESTful"?

Arquitectura organizada por recursos (sustantivos) y métodos HTTP (verbos) para la acción: GET/POST/PUT/PATCH/DELETE. Stateless: cada petición trae toda la información necesaria.

¿Qué es un DTO y por qué no exponen las entidades JPA directamente?

Objeto que define exactamente qué datos viajan entre frontend y API, separado de la entidad JPA, evitando filtrar campos sensibles o arrastrar relaciones innecesarias.

¿Qué es un ORM y qué es el problema N+1?

Hibernate traduce objetos Java a filas SQL. N+1: cargar una lista y hacer una consulta extra por cada ítem. PetCore lo mitiga con `hibernate.default_batch_fetch_size=50`.

¿Qué es Maven y para qué sirve el pom.xml?

Gestor de dependencias y build de Java. El `pom.xml` declara las ~16 dependencias directas con su versión exacta.

¿Qué significa "stateless" y por qué es importante aquí?

El servidor no guarda sesión entre peticiones — el JWT en la cookie trae todo lo necesario. Clave porque Heroku puede reiniciar el dyno sin perder sesiones activas.

¿Qué es la idempotencia y por qué GET y DELETE la necesitan?

Hacer la operación una o cien veces produce el mismo resultado final. El interceptor de reintentos de PetCore solo reintenta GET/HEAD, nunca POST, para no duplicar registros.

¿Qué es HTTPS/TLS y por qué importa para las cookies?

Cifra la comunicación en tránsito. La cookie `authToken` tiene el atributo `Secure`: nunca viaja por HTTP sin cifrar.

¿Qué es un índice de base de datos (IX_)?

Estructura auxiliar para encontrar filas rápido sin recorrer toda la tabla. PK/UQ generan índices automáticos; `IX_` se reserva para índices adicionales en columnas muy consultadas.

¿Qué es una transacción y qué significa ACID?

Agrupa operaciones para que se completen todas o ninguna. ACID: Atomicidad, Consistencia, Aislamiento, Durabilidad. Spring lo maneja con `@Transactional`.

¿Por qué Argon2 y no bcrypt?

Ambos diseñados para hashear contraseñas lentamente, pero Argon2 (ganador PHC 2015) es costoso también en memoria, más resistente a ataques con GPUs/ASICs.

¿Qué es VAPID en las notificaciones push?

Estándar para que el servidor se identifique ante los servicios push del navegador. PetCore expone la clave pública en un endpoint abierto y firma cada notificación con la privada.

¿Capacitor vs. React Native vs. Flutter?

Flutter/React Native reescriben la interfaz con su propio framework. Capacitor reutiliza el 100% del frontend web existente dentro de un WebView nativo con plugins.

SECCIÓN 8 · CRITERIO 10 · 42 PREGUNTAS POR DIFICULTAD

Banco de preguntas para la defensa externa

Organizado por criterio de la rúbrica y por dificultad (🟢 fácil, 🟡 intermedia, 🔴 difícil). Practiquen primero las fáciles.

1. Documentación del sistema

FÁCIL **¿Qué tipo de documentación tiene el proyecto?**

Manual de usuario (capturas paso a paso) + documentación técnica (instalación/ejecución) + diagramas (casos de uso, entidad-relación).

FÁCIL **¿Dónde pueden encontrar el manual de usuario?**

Indica la ubicación real de tu equipo — prepárala con anticipación.

INTERMEDIA **¿Por qué separar documentación técnica de documentación de usuario?**

Audiencias distintas: el personal necesita saber cómo usar el sistema, un desarrollador necesita saber cómo está construido.

INTERMEDIA **¿Qué diagramas incluyeron y qué muestra cada uno?**

Casos de uso (base del criterio 8), entidad-relación (las 50 tablas), y opcionalmente arquitectura general.

DIFÍCIL **Si un desarrollador nuevo se uniera al equipo mañana, ¿qué necesitaría leer primero?**

1) instalación técnica, 2) arquitectura general, 3) modelo entidad-relación, 4) un módulo completo de ejemplo.

2-3. Diseño de interfaces (web y móvil)

FÁCIL **¿Qué significa que un diseño sea "responsive"?**

Se adapta automáticamente al tamaño de pantalla sin que el usuario tenga que hacer zoom o scroll horizontal.

FÁCIL **¿Qué framework CSS usaron y por qué?**

Bootstrap 5.3.8, por su grid y utilidades ya probadas, complementado con CSS propio.

INTERMEDIA **¿Cómo decide el sistema sidebar fija vs. menú hamburguesa?**

Combina `window.innerWidth` y `navigator.userAgent` en JavaScript.

INTERMEDIA **¿Diferencias de UX entre dashboard de Administrador y Farmacéutico?**

Administrador prioriza indicadores generales; Farmacéutico prioriza el POS para velocidad operativa.

DIFÍCIL **¿Cómo garantizan consistencia visual entre 60+ páginas sin React?**

Variables CSS compartidas, patrón HTML repetido por tipo de página, y un controller común que genera sidebar/navbar en tiempo de ejecución.

4-5. Funcionalidad del sistema (CRUD web y móvil)

FÁCIL **¿Qué significa CRUD?**

Crear, Leer, Actualizar, Eliminar.

FÁCIL **Demuestre crear, editar y eliminar un registro en vivo**

Módulo simple (ej. Especie): crear → leer listado → actualizar → eliminar, narrando el fetch en cada paso.

INTERMEDIA **¿Qué pasa si el backend está caído al guardar algo?**

Reintentos automáticos con backoff creciente (1.5s-8s) ante 502/503/504; si persiste, alerta clara al usuario.

INTERMEDIA **¿Cómo manejan la paginación en listados grandes?**

Endpoints paginados (`PaginacionUtils.java`) que devuelven un bloque de resultados por vez.

DIFÍCIL **¿Qué pasa si dos usuarios editan el mismo registro al mismo tiempo?**

Actualmente "último que guarda, gana" (sin bloqueo optimista) — mejora identificada: agregar `@Version` de JPA.

DIFÍCIL **¿Qué pasaría si eliminan una mascota con expedientes clínicos?**

Las FK lo impiden; el backend responde 409 en vez de dejar expedientes huérfanos. Por eso se usa baja lógica, no borrado físico.

6. Validaciones e integridad de datos

FÁCIL **¿Qué es una validación de datos?**

Verificar que un dato cumple el formato/tipo esperado antes de guardarlo.

FÁCIL **Dé un ejemplo real de un campo validado**

El DUI: formato `00000000-0`, validado con regex en el DTO y con CHECK en Oracle.

INTERMEDIA **¿Diferencia entre validar en frontend y en backend?**

Frontend da feedback inmediato pero se puede saltar; el backend garantiza la integridad real.

INTERMEDIA **¿Qué pasa si suben un archivo que no es imagen?**

`accept="image/*"` en el frontend es solo sugerencia; Cloudinary/backend verifican el tipo MIME real y el límite de 10MB.

DIFÍCIL **¿Cómo evitan dosis fuera de rango en una prescripción sin hardcodear reglas?**

`TBL_POSOLOGIA_FARMACO` define el rango de referencia por producto; agregar un fármaco nuevo no requiere tocar la lógica de validación.

7. Autenticación y roles de usuario

FÁCIL **¿Cuántos roles hay y cuáles son?**

Cinco: Administrador, Veterinario, Recepcionista, Farmacéutico y Propietario.

FÁCIL **¿Qué pasa si escribo mal la contraseña?**

401 genérico, sin especificar si falló el correo o la contraseña, por seguridad.

INTERMEDIA **¿Qué es un hash y por qué no guardan la contraseña tal cual?**

Función de un solo sentido; si roban la BD, no pueden recuperar la contraseña original.

INTERMEDIA **¿Qué es Argon2 y por qué lo eligieron?**

Ganador de la Password Hashing Competition 2015, costoso en memoria, resistente a fuerza bruta con hardware especializado.

DIFÍCIL **Si el SMTP se cae, ¿qué le pasa a la recuperación de contraseña?**

El flujo queda bloqueado en ese paso — dependencia externa reconocida; mejora posible: canal alterno asistido por Administrador.

8. Protección de endpoints y vistas

FÁCIL **¿Qué pasa si entro a una URL sin sesión?**

El guard redirige automáticamente al login, sin mostrar el contenido ni un instante.

INTERMEDIA **¿Diferencia entre 401 y 403?**

401: no sé quién eres. 403: sé quién eres, pero no tienes permiso.

INTERMEDIA **¿Qué es el bfcache y cómo lo bloquean?**

Caché de restauración del navegador; `pageshow` + `event.persisted` fuerza recarga y re-verificación.

DIFÍCIL **Si roban la cookie httpOnly directamente del tráfico de red, ¿qué pasa?**

httpOnly protege de XSS, no de interceptación de red — contra eso protegen HTTPS/TLS y el atributo Secure.

DIFÍCIL **¿Por qué el endpoint de WebSocket es público?**

Debilidad reconocida: el handshake no valida JWT todavía; impacto limitado porque no transmite datos sensibles por usuario.

9. Despliegue

FÁCIL **¿Dónde está alojado cada componente?**

Frontend en Vercel, API en Heroku, base de datos en Oracle Cloud, app móvil en un Android físico.

INTERMEDIA **¿Qué es un "cold start" en Heroku?**

El dyno "duerme" sin tráfico; la primera petición tarda más. Mitigado con reintentos y un ping periódico de GitHub Actions.

INTERMEDIA **¿Cómo se conectan Vercel, Heroku y Oracle entre sí?**

Solo por configuración: URLs públicas, CORS y credenciales JDBC — son tres servicios totalmente independientes.

DIFÍCIL **Si Heroku reinicia el dyno, ¿qué garantiza que no se pierdan las sesiones?**

El diseño stateless: la sesión vive en el JWT del navegador, no en memoria del servidor.

10. Dominio teórico y funcional

FÁCIL **¿Cuántos módulos y tablas tiene el sistema?**

51 módulos backend y 50 tablas Oracle, en 6 dominios funcionales.

INTERMEDIA **¿Cuál fue la parte más difícil de construir en equipo?**

Coordinar "deny by default" en 51 módulos y sincronizar CORS entre HTTP y WebSocket.

INTERMEDIA **¿Cómo se dividió el trabajo entre los 6 integrantes?**

Ajusta a tu equipo — todos deben poder explicar su parte y, a grandes rasgos, el resto del sistema.

DIFÍCIL **Si tuvieran más tiempo, ¿qué mejorarían?**

Securizar el handshake de `/ws`, agregar bloqueo optimista por versión, cacheo de assets en el Service Worker.

DIFÍCIL **¿Por qué Oracle y no MySQL/PostgreSQL?**

Requisito institucional; `GenerationType.IDENTITY` compatible con el autoincremento de Oracle.

DIFÍCIL **¿Qué cambiarían para escalar a 50 clínicas?**

Multi-tenancy, balanceo de carga, particionamiento de tablas grandes.

DIFÍCIL **¿Por qué arquitectura modular por feature y no por capas?**

Con 51 entidades, por capas sería inmanejable; por feature agrupa todo de una entidad en una carpeta, facilitando trabajo en equipo.

SECCIÓN 9 · ANTES DE LOS GUIONES

Logística del equipo: 6 integrantes, 4 días, 2 personas por turno

ExpoTécnica dura 4 días con turno matutino y vespertino cada día (8 turnos en total), pero solo **2 integrantes** están presentes en cada turno. Por eso **cualquier pareja de los 6 debe poder ejecutar ambos guiones completos**, no solo su parte.

Calendario de turnos (complétenlo como equipo)

DÍA	TURNO MATUTINO — 2 INTEGRANTES	TURNO VESPERTINO — 2 INTEGRANTES
Día 1	_____ · _____	_____ · _____
Día 2	_____ · _____	_____ · _____
Día 3	_____ · _____	_____ · _____
Día 4	_____ · _____	_____ · _____

> Recomendación: mezclen en cada turno a alguien fuerte en backend/base de datos con alguien fuerte en frontend/app móvil. Y practiquen **los 6** el guion del jurado completo al menos una vez, sin importar qué parte dominaron durante el desarrollo.

SECCIÓN 10 · GUION 1 DE 2

⚖️ Guion para el Jurado Externo

Para **2 presentadores** intercambiables (cualquier pareja del equipo). Sin límite de tiempo formal — prioricen claridad sobre velocidad.

> ⚠️ Antes de empezar, en cualquiera de los 8 turnos: verifiquen que Vercel está activo, que la API de Heroku ya está "despierta", y que la app móvil está instalada y probada en el dispositivo físico (ver Sección 2 de este documento).

1. Apertura y presentación del equipo

~2 min

Presentador 1: "Somos parte del equipo PetCore. Les presentamos un sistema integral de gestión veterinaria, con Java/Spring Boot en el backend, HTML/CSS/Bootstrap/JS sin frameworks en el frontend, Oracle, y una app móvil con Capacitor."

Presentador 2: "El sistema cubre todo el flujo real de una clínica, con cuatro roles de personal y un quinto rol de Propietario para la app móvil."

Pueden mencionar: "somos un equipo de 6; hoy nos corresponde a nosotros dos, pero todo el equipo domina el sistema completo."

2. Documentación del sistema

~2 min

Mostrar manual de usuario y documentación técnica. Señalar el diagrama de casos de uso: "muestra, por cada rol, qué puede hacer — es la base de la protección de endpoints."

3. Demo del sistema web (Vercel)

~8 min

URL pública en vivo: login como Administrador → recorrido del dashboard → CRUD completo narrando validación frontend y backend → responsive si es posible → cambio de rol mostrando dashboards distintos.

4. Protección de endpoints y vistas

~5 min

En vivo: 1) endpoint público, 2) endpoint protegido sin sesión → login, 3) rol equivocado → acceso denegado, 4) botón "Atrás" tras logout → recarga y pide login de nuevo.

5. Demo de la API (Heroku)

~5 min

Postman/consola: endpoint público (200) vs. protegido sin cookie (401). Explicar `ApiResponse`. Mencionar CORS.

6. Demo de la base de datos (Oracle Cloud)

~5 min

SQL Developer: modelo de tablas, consulta reflejando el dato recién creado, ejemplo de CHECK CONSTRAINT.

7. Demo de la app móvil (Android físico)

~6 min

Login como propietario (con Google si aplica), mascotas, CRUD simple, notificación push si es posible. Mencionar que es el mismo frontend empaquetado con Capacitor.

8. Arquitectura y dominio técnico profundo

Sin límite

Resumen de arquitectura modular (51 módulos), JWT en cookie httpOnly, patrón "deny by default". A partir de aquí el jurado pregunta libremente — repasar el banco de preguntas y conceptos clave de este documento.

9. Cierre

~1 min

"Con esto cubrimos el sistema web, la API, la base de datos y la app móvil de PetCore, todos en producción y funcionando de forma integrada. Quedamos atentos a sus preguntas." Agradecer.

SECCIÓN 11 · GUION 2 DE 2

📣 Guion para el Público (resumido)

Para visitantes del stand durante los 4 días. Lenguaje simple, 3-5 minutos, repetible por 1-2 personas muchas veces sin cansarse.

1. Gancho inicial

~20 seg

"¡Hola! ¿Tienen mascota? Les mostramos en 3 minutos cómo funciona PetCore, un sistema para que una clínica veterinaria maneje todo digitalmente: citas, historial médico, inventario y ventas — y también hay una app para que el dueño vea todo desde su celular."

2. El problema que resuelve

~30 seg

"Muchas clínicas aún llevan el control en papel o Excel. Eso hace lento atender y fácil perder información importante, como una alergia a un medicamento. PetCore centraliza todo en un solo sistema."

3. Demo rápida en vivo

~2 min

Mostrar solo lo visual: agendar/mostrar una cita, expediente con foto, un vistazo a la app móvil, el punto de venta si hay interés. Dejar que la persona toque la pantalla si parece interesada.

4. Lo que lo hace especial

~30 seg

"Lo construimos nosotros mismos, desde la web hasta la app y la base de datos — y está funcionando de verdad en internet ahora mismo."

5. Cierre y despedida

~20 seg

"Eso es PetCore en resumen. ¡Gracias por pasar! Si tienen alguna pregunta con gusto se la resolvemos."

> Tip para los 4 días: este guion es corto a propósito — se va a repetir muchas veces por turno. Si quien lo da se lo sabe de memoria, suena natural en la repetición 20 igual que en la 1.